AAI 627 Homework 2
Factor Model & Time Series Analysis

Jude Eschete

February 22, 2026

1 Part I: Factor and Industry Return Analysis

The dataset `EE627A_HW1_Data.csv` contains 948 monthly observations (January 1927 – December 2005) of returns on 30 industry portfolios and five factor series: Market excess return (Mkt-RF), SMB, HML, the risk-free rate (RF), and Momentum (Mom).

1.1 1.1.1 Correlation Matrix

Table 1 presents the correlation of each industry return with the four-factor model variables and RF. Table 2 shows the inter-factor correlation matrix.

Table 1: Correlation of Four Factors + RF with 30 Industry Portfolios

Industry	**Mkt-RF**	**SMB**	**HML**	**RF**	**Mom**
Food	0.8359	0.2017	0.2151	0.0322	−0.2803
Beer	0.7077	0.3510	0.2150	−0.0113	−0.2001
Smoke	0.5843	0.1032	0.1718	0.0630	−0.2192
Games	0.8302	0.4121	0.2504	−0.0250	−0.3570
Books	0.8301	0.4081	0.2506	0.0092	−0.3064
Hshld	0.8162	0.2619	0.1074	−0.0242	−0.1997
Clths	0.7806	0.4601	0.2467	−0.0202	−0.3605
Hlth	0.8040	0.2089	0.0644	0.0078	−0.2258
Chems	0.8839	0.2080	0.1924	−0.0339	−0.3012
Txtls	0.8234	0.4954	0.3475	−0.0357	−0.3711
Cnstr	0.9136	0.4015	0.2472	−0.0313	−0.3232
Steel	0.8679	0.3631	0.3437	−0.0488	−0.3945
FabPr	0.9250	0.4068	0.2603	−0.0567	−0.3724
ElcEq	0.9041	0.2574	0.2024	−0.0147	−0.3246
Autos	0.8433	0.2860	0.3028	−0.0599	−0.3893
Carry	0.8324	0.3626	0.3374	−0.0336	−0.3438
Mines	0.7062	0.3439	0.2457	−0.0522	−0.2712
Coal	0.4905	0.2673	0.1722	−0.0296	−0.1371
Oil	0.7726	0.1377	0.3023	−0.0240	−0.2499
Util	0.7618	0.1751	0.3547	0.0267	−0.3317
Telcm	0.7552	0.1606	0.0991	0.0344	−0.2939
Servs	0.4956	0.2851	−0.0849	−0.0115	−0.0925
BusEq	0.8426	0.3298	−0.0663	−0.0516	−0.2284
Paper	0.8691	0.2757	0.1973	−0.0243	−0.3002
Trans	0.8587	0.3481	0.4305	−0.0179	−0.4232
Whlsl	0.7960	0.4598	0.2092	−0.0129	−0.2604
Rtail	0.8587	0.3029	0.1150	−0.0091	−0.3094
Meals	0.7539	0.3686	0.1636	−0.0230	−0.2508
Fin	0.9159	0.2826	0.3246	−0.0116	−0.4204
Other	0.8420	0.4239	0.1651	−0.0357	−0.2903

Table 2: Inter-Factor Correlation Matrix

	Mkt-RF	**SMB**	**HML**	**RF**	**Mom**
Mkt-RF	1.0000	0.3269	0.2161	−0.0687	−0.3383
SMB	0.3269	1.0000	0.0941	−0.0596	−0.1640
HML	0.2161	0.0941	1.0000	0.0121	−0.4006
RF	−0.0687	−0.0596	0.0121	1.0000	0.0391
Mom	−0.3383	−0.1640	−0.4006	0.0391	1.0000

1.2 1.1.2 Most Highly Correlated Factor per Industry

Among the four factors (Mkt-RF, SMB, HML, and Momentum), the **market excess return (Mkt-RF)** correlates most highly with *every* one of the 30 industry portfolios. The correlations range from 0.4905 (Coal) to 0.9250 (Fabricated Products), with the vast majority exceeding 0.75. This is consistent with the Capital Asset Pricing Model (CAPM): the market factor is the dominant driver of systematic risk across all industries.

1.3 1.1.3 Negative Correlations & RF Analysis

Factor with negative correlation to every industry. Momentum (Mom) is the only factor that is negatively correlated with all 30 industry portfolios (correlations ranging from −0.0925 for Services to −0.4232 for Transportation). The remaining factors — Mkt-RF, SMB, and HML — are positively correlated with all or nearly all industries (HML is slightly negative for Services and Business Equipment only).

Does RF correlate highly with the 30 industry time series? No. The risk-free rate (RF) exhibits extremely weak correlations with all 30 industries: the absolute correlations range from 0.0078 to 0.0630, with a mean of −0.0175. None of the absolute correlations exceed 0.10, let alone the 0.30 or 0.50 thresholds typically considered "moderate" or "strong." This indicates that the risk-free rate has essentially **no linear relationship** with industry returns.

1.4 1.2.1 Autocorrelation Function (ACF), Lags 1–10

The ACF values for the four-factor time series are computed for lags 1 through 10. With $n = 948$ observations, the approximate 95% confidence bound is $\pm 1.96/\sqrt{948} \approx \pm 0.0637$.

Table 3: Autocorrelation Function (ACF) for Four Factors, Lags 1–10

Lag	**Mkt-RF**	**SMB**	**HML**	**Mom**
1	0.1072^*	0.0753^*	0.1780^*	0.0578
2	−0.0163	0.0592	−0.0133	-0.0774^*
3	-0.1082^*	−0.0541	−0.0316	-0.0745^*
4	0.0056	−0.0316	-0.0805^*	−0.0492
5	0.0701^*	−0.0538	−0.0614	−0.0390
6	−0.0201	0.0099	0.0078	0.0511
7	0.0126	0.0226	0.0645^*	−0.0362
8	0.0367	0.0264	−0.0022	−0.0159
9	0.0817^*	0.0836^*	0.1149^*	0.0122
10	0.0190	0.0249	0.0261	−0.0441

* Significant at 95% level ($|\text{ACF}| > 0.0637$).

1.5 1.2.2 AR(1) Model Assessment

An AR(1) model posits $X_t = c + \phi X_{t-1} + \varepsilon_t$. Table 4 summarizes the fitted AR(1) coefficients for each factor series and whether the lag-1 coefficient ϕ is statistically significant. The Ljung-Box test on residuals checks whether the AR(1) specification adequately captures the serial dependence.

Table 4: AR(1) Model Results for Four Factors

Factor	$\hat{\phi}$	**p-value**	ϕ **Significant?**	**Residuals WN?**
Mkt-RF	0.1072	9.10×10^{-4}	Yes	No
SMB	0.0754	2.01×10^{-2}	Yes	Yes
HML	0.1780	2.52×10^{-8}	Yes	No
Mom	0.0578	7.48×10^{-2}	No	No

Observations.

- **Mkt-RF:** The lag-1 coefficient $\hat{\phi} = 0.1072$ is statistically significant ($p < 0.001$), indicating mild positive autocorrelation. However, the Ljung-Box test rejects white noise for the residuals at several lags, meaning a simple AR(1) captures some but not all of the serial structure. An AR(1) is a reasonable starting model, but higher-order terms may improve the fit.

- **SMB:** The lag-1 coefficient $\hat{\phi} = 0.0754$ is significant at the 5% level ($p = 0.020$), and the Ljung-Box residual test shows no significant autocorrelation at any lag. This makes SMB the **best candidate for an AR(1) model** among the four factors — the AR(1) adequately captures all serial dependence.

- **HML:** The lag-1 coefficient $\hat{\phi} = 0.1780$ is highly significant ($p < 10^{-7}$), the strongest first-order autocorrelation of any factor. However, the residuals exhibit significant autocorrelation at lags 9–10, so the AR(1) model leaves some serial structure unaccounted for. A longer AR or ARMA model may be needed.

- **Momentum (Mom):** The lag-1 coefficient $\hat{\phi} = 0.0578$ is *not* significant at the 5% level ($p = 0.075$), suggesting that this series does not exhibit meaningful first-order autocorrelation. Additionally, the Ljung-Box test indicates residual autocorrelation at higher lags. Therefore, an AR(1) model is **not appropriate** for the Momentum factor.

Summary. Three of the four factor series (Mkt-RF, SMB, HML) have statistically significant lag-1 autocorrelation, supporting some degree of AR(1) behavior. However, only **SMB** is fully and cleanly described by an AR(1) model (significant ϕ with white-noise residuals). For Mkt-RF and HML, additional lags or more complex ARMA structures would better capture the remaining serial dependence. Momentum shows no significant AR(1) structure.

2 Part II: Time Series Model Calculations

Given

$$X_t = 0.01 + 0.2X_{t-2} + a_t,$$

where $a_t \sim WN(0, 0.02)$.

2.1 2.1 Mean and Variance of X_t

For a stationary process, let $\mu = E[X_t]$. Taking expectation:

$$\mu = 0.01 + 0.2\mu \quad \Rightarrow \quad \mu = \frac{0.01}{1 - 0.2} = 0.0125.$$

Define $Y_t = X_t - \mu$. Then

$$Y_t = 0.2Y_{t-2} + a_t.$$

Let $\gamma_k = \mathrm{Cov}(Y_t, Y_{t-k})$ and $\sigma_a^2 = 0.02$. Yule-Walker equations give:

$$\gamma_1 = 0, \qquad \gamma_2 = 0.2\gamma_0,$$

$$\gamma_0 = 0.2\gamma_2 + \sigma_a^2 = 0.2(0.2\gamma_0) + 0.02.$$

So

$$0.96\gamma_0 = 0.02 \Rightarrow \gamma_0 = \frac{0.02}{0.96} = 0.0208333.$$

Hence

$$E[X_t] = 0.0125, \qquad \mathrm{Var}(X_t) = 0.0208333.$$

2.2 2.2 Lag-1 and Lag-2 Autocorrelations

By definition $\rho_k = \gamma_k/\gamma_0$:

$$\rho_1 = \frac{\gamma_1}{\gamma_0} = 0, \qquad \rho_2 = \frac{\gamma_2}{\gamma_0} = 0.2.$$

2.3 2.3 One- and Two-Step Forecasts at $t = 100$

Given $X_{100} = -0.01$ and $X_{99} = 0.02$:

$$\hat{X}_{101|100} = E[X_{101} \mid \mathcal{F}_{100}] = 0.01 + 0.2X_{99} = 0.01 + 0.2(0.02) = 0.014.$$

$$\hat{X}_{102|100} = E[X_{102} \mid \mathcal{F}_{100}] = 0.01 + 0.2X_{100} = 0.01 + 0.2(-0.01) = 0.008.$$

Therefore, the forecasts are:

$$\hat{X}_{101|100} = 0.014, \qquad \hat{X}_{102|100} = 0.008.$$

3 Appendix: Python Code

```python
"""
AAI 627 Homework 2 - Jude Eschete
Higgs Boson Machine Learning Challenge / Factor Model Analysis

This script performs the following analyses on EE627A_HW1_Data.csv:
  1.1.1 Correlation matrix between all time series
  1.1.2 Which factor correlates most highly with every industry
  1.1.3 Which factor correlates negatively with every industry; RF correlation with industries
  1.2.1 ACF for time-lag 1..10 for the four-factor time series
  1.2.2 AR(1) model assessment for the 4 factor time series
"""

import os
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import acf
from statsmodels.tsa.ar_model import AutoReg
from statsmodels.stats.diagnostic import acorr_ljungbox

# -- Paths -------------------------------
SCRIPT_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_FILE = os.path.join(SCRIPT_DIR, "EE627A_HW1_Data.csv")
OUTPUT_FILE = os.path.join(SCRIPT_DIR, "HW2_Results.txt")
OUTPUT_FILE_PART2 = os.path.join(SCRIPT_DIR, "HW2_Part2_Results.txt")

# -- Load data -------------------------------
df = pd.read_csv(DATA_FILE)
df.set_index("Date", inplace=True)

# Column groupings
factors = ["Mkt-RF", "SMB", "HML", "Mom"]          # four-factor model
all_factors = ["Mkt-RF", "SMB", "HML", "RF", "Mom"]  # includes risk-free
industries = [c for c in df.columns if c not in all_factors]

print(f"Loaded {len(df)} monthly observations, {len(industries)} industries, {len(all_factors)} factor columns.")

# -- Helper: write and print -------------------------------
lines = []

def out(text=""):
    lines.append(text)
    print(text)

```

```python

# ==================================================
# 1.1.1  Full correlation matrix
# ==================================================
out("=" * 90)
out("1.1.1  CORRELATION MATRIX (all time series)")
out("=" * 90)

corr_all = df.corr()

# Print correlation matrix in a readable format (factor columns vs industries)
out("\nCorrelation of Four Factors + RF with 30 Industries:\n")
header = f"{'Industry':<10}" + "".join(f"{f:>10}" for f in all_factors)
out(header)
out("-" * len(header))
for ind in industries:
    row = f"{ind:<10}" + "".join(f"{corr_all.loc[ind, f]:>10.4f}" for f in all_factors)
    out(row)

# Also show inter-factor correlations
out("\nInter-factor correlation matrix:\n")
factor_corr = df[all_factors].corr()
header2 = f"{'':>10}" + "".join(f"{f:>10}" for f in all_factors)
out(header2)
out("-" * len(header2))
for f1 in all_factors:
    row = f"{f1:>10}" + "".join(f"{factor_corr.loc[f1, f2]:>10.4f}" for f2 in all_factors)
    out(row)

# ==================================================
# 1.1.2  Which factor correlates most highly with every industry?
# ==================================================
out("\n" + "=" * 90)
out("1.1.2  FACTOR WITH HIGHEST CORRELATION FOR EACH INDUSTRY")
out("=" * 90 + "\n")

out(f"{'Industry':<10}{'Most Corr. Factor':>20}{'Correlation':>15}")
out("-" * 45)
most_corr_summary = {}
for ind in industries:
    corrs = {f: corr_all.loc[ind, f] for f in factors}
    best_f = max(corrs, key=lambda k: abs(corrs[k]))
    most_corr_summary[ind] = (best_f, corrs[best_f])
    out(f"{ind:<10}{best_f:>20}{corrs[best_f]:>15.4f}")
```

```python

# Tally
from collections import Counter
tally = Counter(v[0] for v in most_corr_summary.values())
out("\nSummary count of most-correlated factor across 30 industries:")
for f, cnt in tally.most_common():
    out(f"  {f}: {cnt} industries")

# ==================================================
# 1.1.3  Which factor correlates negatively with every industry?
#        Does RF correlate highly with the 30 industry time series?
# ==================================================
out("\n" + "=" * 90)
out("1.1.3  FACTORS WITH NEGATIVE CORRELATION TO EVERY INDUSTRY  &  RF ANALYSIS")
out("=" * 90 + "\n")

# Check each factor: is correlation negative for ALL 30 industries?
for f in factors:
    neg_all = all(corr_all.loc[ind, f] < 0 for ind in industries)
    neg_count = sum(1 for ind in industries if corr_all.loc[ind, f] < 0)
    out(f"  {f:>8}: Negative with ALL industries? {'YES' if neg_all else 'NO'}  "
        f"(negative with {neg_count}/{len(industries)} industries)")

# RF analysis
out(f"\n  {'RF':>8}: Correlation with each industry:")
rf_corrs = {ind: corr_all.loc[ind, "RF"] for ind in industries}
out(f"{'Industry':<10}{'Corr with RF':>15}")
out("-" * 25)
for ind in industries:
    out(f"{ind:<10}{rf_corrs[ind]:>15.4f}")

rf_vals = list(rf_corrs.values())
out(f"\n  RF correlation stats across 30 industries:")
out(f"    Mean : {np.mean(rf_vals):.4f}")
out(f"    Min  : {np.min(rf_vals):.4f}")
out(f"    Max  : {np.max(rf_vals):.4f}")
out(f"    |corr| > 0.5 for {sum(1 for v in rf_vals if abs(v) > 0.5)} / {len(industries)} industries")
out(f"    |corr| > 0.3 for {sum(1 for v in rf_vals if abs(v) > 0.3)} / {len(industries)} industries")

rf_neg_all = all(v < 0 for v in rf_vals)
out(f"    Negative with ALL industries? {'YES' if rf_neg_all else 'NO'}")
out(f"  => RF does {'NOT ' if np.mean(np.abs(rf_vals)) < 0.3 else ''}correlate highly with the 30 industry time series.")
```

```python

# ==================================================
# 1.2.1  Autocorrelation function (ACF) for lags 1-10, four-factor series
# ==================================================
out("\n" + "=" * 90)
out("1.2.1  AUTOCORRELATION FUNCTION (ACF) - LAGS 1 TO 10")
out("=" * 90 + "\n")

acf_results = {}
n = len(df)
# Bartlett approximate 95% CI bound
ci_bound = 1.96 / np.sqrt(n)
out(f"Number of observations: {n}")
out(f"Approximate 95% confidence bound: +/- {ci_bound:.4f}\n")

header3 = f"{'Lag':>5}" + "".join(f"{f:>12}" for f in factors)
out(header3)
out("-" * len(header3))
for lag in range(1, 11):
    row = f"{lag:>5}"
    for f in factors:
        acf_vals = acf(df[f], nlags=10, fft=True)
        if lag == 1:
            acf_results[f] = acf_vals[1:]  # store lags 1-10
        row += f"{acf_vals[lag]:>12.4f}"
    out(row)

out(f"\n(Values outside +/- {ci_bound:.4f} are statistically significant at 95% level)")

# Mark significant lags
out("\nSignificant ACF values (|ACF| > 95% bound):")
for f in factors:
    acf_vals = acf(df[f], nlags=10, fft=True)
    sig_lags = [k for k in range(1, 11) if abs(acf_vals[k]) > ci_bound]
    if sig_lags:
        out(f"  {f}: lags {sig_lags}")
    else:
        out(f"  {f}: none")

# ==================================================
# 1.2.2  AR(1) model assessment
# ==================================================
out("\n" + "=" * 90)
out("1.2.2  AR(1) MODEL ASSESSMENT FOR THE FOUR FACTOR TIME SERIES")
out("=" * 90 + "\n")

```

```python
out("An AR(1) process: X_t = c + phi * X_{t-1} + epsilon_t")
out("We fit AR(1) to each factor and check:\n"
    "  (a) significance of lag-1 coefficient (phi)\n"
    "  (b) Ljung-Box test on residuals for remaining autocorrelation\n")

for f in factors:
    out(f"--- {f} ---")
    series = df[f].values
    model = AutoReg(series, lags=1, old_names=False)
    result = model.fit()
    phi = result.params[1]
    pval_phi = result.pvalues[1]
    const = result.params[0]
    resid = result.resid

    # Ljung-Box test on residuals (lags 1-10)
    lb_test = acorr_ljungbox(resid, lags=10, return_df=True)
    lb_pvals = lb_test["lb_pvalue"].values

    acf1 = acf(series, nlags=1, fft=True)[1]

    out(f"  ACF(1)            = {acf1:.4f}")
    out(f"  AR(1) constant    = {const:.4f}")
    out(f"  AR(1) phi         = {phi:.4f}  (p-value = {pval_phi:.4e})")
    out(f"  phi significant?  {'YES' if pval_phi < 0.05 else 'NO'} (at 5% level)")
    out(f"  Ljung-Box p-values on residuals (lags 1-10):")
    for lag_idx in range(10):
        sig_marker = " *" if lb_pvals[lag_idx] < 0.05 else ""
        out(f"    Lag {lag_idx+1:>2}: p = {lb_pvals[lag_idx]:.4f}{sig_marker}")
    residual_ok = all(p > 0.05 for p in lb_pvals)
    out(f"  Residuals white noise? {'YES' if residual_ok else 'NO'}")

    is_ar1 = pval_phi < 0.05
    out(f"  => AR(1) model {'IS' if is_ar1 else 'is NOT'} appropriate for {f}.\n")

# Summary
out("\n--- AR(1) SUMMARY ---")
for f in factors:
    series = df[f].values
    model = AutoReg(series, lags=1, old_names=False)
    result = model.fit()
    phi = result.params[1]
    pval_phi = result.pvalues[1]
    is_ar1 = pval_phi < 0.05
```

```python
    out(f"  {f:>8}: phi = {phi:.4f}, p = {pval_phi:.4e} => {'AR(1) YES' if is_ar1 else 'AR(1) NO'}")

# ==================================================
# Write output file
# ==================================================
with open(OUTPUT_FILE, "w", encoding="utf-8") as fout:
    fout.write("\n".join(lines))

print(f"\n*** Results written to {OUTPUT_FILE} ***")


# ==================================================
# Part II: AR(2)-style model calculations
# X_t = 0.01 + 0.2 * X_{t-2} + a_t,    a_t ~ WN(0, 0.02)
# ==================================================
part2_lines = []

def out2(text=""):
    part2_lines.append(text)
    print(text)


out2("=" * 90)
out2("PART II  MODEL: X_t = 0.01 + 0.2 X_{t-2} + a_t,   a_t ~ WN(0, 0.02)")
out2("=" * 90)

# Given model parameters
c0 = 0.01
phi2 = 0.2
sigma_a2 = 0.02

# 2.1 Mean and variance
mu = c0 / (1 - phi2)

# For centered process Y_t = X_t - mu: Y_t = phi2 * Y_{t-2} + a_t
# Yule-Walker: gamma2 = phi2*gamma0, gamma0 = phi2*gamma2 + sigma_a2
# => gamma0 = sigma_a2 / (1 - phi2^2)
gamma0 = sigma_a2 / (1 - phi2**2)
var_x = gamma0

out2("\n2.1 Mean and Variance")
out2(f"  Mean E[X_t] = {mu:.10f}")
out2(f"  Variance Var(X_t) = {var_x:.10f}")

# 2.2 ACF at lags 1 and 2
```

```python
gamma1 = 0.0
gamma2 = phi2 * gamma0
rho1 = gamma1 / gamma0
rho2 = gamma2 / gamma0

out2("\n2.2 Lag-1 and Lag-2 autocorrelations")
out2(f"  rho(1) = {rho1:.10f}")
out2(f"  rho(2) = {rho2:.10f}")

# 2.3 Forecasts at origin t=100
x_100 = -0.01
x_99 = 0.02

# 1-step ahead: X_101 = c0 + phi2*X_99 + a_101, E[a_101|F_100]=0
xhat_101 = c0 + phi2 * x_99

# 2-step ahead: X_102 = c0 + phi2*X_100 + a_102, E[a_102|F_100]=0
xhat_102 = c0 + phi2 * x_100

out2("\n2.3 Forecasts at t=100 (given X_100=-0.01, X_99=0.02)")
out2(f"  1-step forecast  Xhat_101|100 = {xhat_101:.10f}")
out2(f"  2-step forecast  Xhat_102|100 = {xhat_102:.10f}")

with open(OUTPUT_FILE_PART2, "w", encoding="utf-8") as fout2:
    fout2.write("\n".join(part2_lines))

print(f"\n*** Part II results written to {OUTPUT_FILE_PART2} ***")
```

Listing 1: AAI627_HW2_Eschete.py